

Preparation aux Entretiens

Algorithmes & Structures de donnees

Comment aborder un probleme de code,
les 11 patterns essentiels, et les pieges a eviter.

Table des matières

1	Methode : comment coder en entretien	2
2	Hash Map / Dictionnaire	3
3	Hash Set	3
4	Two Pointers (deux pointeurs)	4
5	Sliding Window (fenetre glissante)	4
6	Prefix Sum (somme prefixee)	5
7	Sorting (tri)	6
8	Binary Search (recherche dichotomique)	6
9	Stack (pile LIFO)	7
10	Monotonic Stack (pile monotone)	7
11	Backtracking (retour sur trace)	8
12	Linked List (liste chaine)	9
13	Fiche recapitulative des complexites	10

1. Methode : comment coder en entretien

Ne jamais foncer sur le clavier. Un entretien evalue autant ton *raisonnement* que ta solution. Suis systematiquement ces six etapes :

1. **Comprendre** – reformule l'enonce ; clarifie les contraintes (taille, valeurs, doublons, donnees trieées?).
2. **Exemples** – ecris 1 ou 2 exemples a la main, y compris les cas limites (tableau vide, un seul element).
3. **Force brute** – enonce la solution naive et sa complexite. Tu as toujours une base sur laquelle t'appuyer.
4. **Optimiser** – cherche le pattern : quelle structure de donnees ? quel invariant ? un tri prealable aide-t-il ?
5. **Coder** – ecris proprement en parlant a voix haute ; nomme clairement les variables.
6. **Tester** – deroule le code sur un exemple, verifie les cas limites, corrige.

Regle d'or

Annonce toujours la complexite (temps *et* espace) de ta solution, et propose une piste d'amelioration meme si tu n'as pas le temps de l'implementer.

2. Hash Map / Dictionnaire

Structure cle $\rightarrow$ valeur offrant un acces en $O(1)$ moyen. C'est l'outil le plus rentable en entretien : il transforme souvent un $O(n^2)$ en $O(n)$.

L'idée. Au lieu de re-parcourir tout le tableau a chaque fois qu'on cherche quelque chose, on *memorise au fur et a mesure* ce qu'on a deja vu dans un dictionnaire, puis on le consulte instantanement. On echange de la memoire ($O(n)$) contre du temps : c'est le compromis classique « espace contre vitesse ».

Reconnaitre le pattern. Des que l'enonce parle de *compter*, de *regrouper*, de *trouver une paire / un complement*, ou que ta premiere idee est une double boucle qui compare chaque element a tous les autres, pense hash map : elle supprime presque toujours la boucle interne.

Quand l'utiliser

Compter des frequences ; retrouver un complement deja vu (Two Sum) ; grouper des elements par cle ; memoriser des indices deja rencontres.

Complexite : Temps $O(1)$ moyen | Espace $O(n)$

Problemes types : Two Sum (LC 1), Group Anagrams (LC 49), Top K Frequent (LC 347).

```
def two_sum(nums, target):
    seen = {} # valeur -> indice
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

Piege : en cas de collision de cles, la derniere valeur ecrase la precedente. Reflexis a ce que la cle doit reellement représenter.

3. Hash Set

Ensemble d'elements uniques ; test d'appartenance en $O(1)$ moyen.

L'idée. Un set est un hash map sans valeurs : on ne retient que la *presence* des elements. Il repond en temps constant a une seule question, mais la plus utile : « ai-je deja rencontre cet element ? ». On l'alimente au fil du parcours et on interroge avant d'ajouter.

Reconnaitre le pattern. Mots-cles declencheurs : *doublon*, *unique*, *deja vu*, *visite*. Si tu es tente d'ecrire `if x in ma_liste` a l'interieur d'une boucle (recherche en $O(n)$ a chaque tour, donc $O(n^2)$ au total), remplace la liste par un set et le cout tombe a $O(n)$.

Quand l'utiliser

Tester l'appartenance rapidement ; detecter les doublons ; suivre les elements deja visites (parcours de graphe, cycles).

Complexite : Temps $O(1)$ moyen | Espace $O(n)$

Problemes types : Contains Duplicate (LC 217), Longest Consecutive (LC 128), Happy Number (LC 202).

```
def has_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
    seen.add(x)
```

```
return False
```

Piege : un set ne conserve pas l'ordre d'insertion ; si l'ordre compte, utilise un dictionnaire (ordonne depuis Python 3.7) ou une liste annexe.

4. Two Pointers (deux pointeurs)

Deux indices qui avancent selon une regle. Deux variantes : *opposes* (left/right) ou *memme direction* (lent/rapide).

L'idée. Beaucoup de problemes qui semblent demander deux boucles imbriquees peuvent se resoudre avec deux curseurs qui balaient le tableau en un seul passage. Dans la variante *opposee*, on part des deux extremités d'un tableau *trie* et on rapproche les curseurs selon que la somme est trop grande ou trop petite. Dans la variante *lent/rapide*, un curseur avance plus vite que l'autre (utile pour les cycles ou le milieu d'une liste).

Reconnaitre le pattern. Tableau ou chaine *trie*, recherche d'une *paire* qui verifie une condition, *palindrome*, ou l'envie d'ecrire une double boucle $O(n^2)$ sur des donnees ordonnees : les deux pointeurs ramènent souvent le cout a $O(n)$ et $O(1)$ d'espace.

Quand l'utiliser

Tableau *trie* et recherche d'une paire ; palindrome ou inversion ; detection de cycle et recherche du milieu (lent/rapide).

Complexite : Temps $O(n)$ | Espace $O(1)$

Problemes types : Valid Palindrome (LC 125), Two Sum II (LC 167), 3Sum (LC 15).

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        if s < target:
            left += 1
        else:
            right -= 1
    return []
```

Piege : la variante opposee suppose un tableau *trie*. Si l'enonce ne le garantit pas, il faut trier d'abord (+ $O(n \log n)$).

5. Sliding Window (fenetre glissante)

Maintenir une fenetre `[left, right]` et l'ajuster selon une contrainte, au lieu de recalculer chaque sous-tableau.

L'idée. Verifier tous les sous-tableaux coute $O(n^2)$. La fenetre glissante evite ce gaspillage : quand on avance le bord droit, on *ajoute* un element ; quand la contrainte est violee, on avance le bord gauche pour *retirer* des elements. Chaque element entre et sort de la fenetre au plus une fois, d'ou un cout total de $O(n)$. C'est comme une chenille qui s'etire et se contracte le long du tableau.

Reconnaitre le pattern. L'enonce demande le *plus long* / *plus court* sous-tableau ou sous-chaine *contigu* respectant une condition (somme, nombre de caracteres distincts, etc.), ou une fenetre de *taille fixe k*. Le mot cle est **contigu** : si les elements doivent etre voisins, pense fenetre glissante.

Quand l'utiliser

Sous-tableau ou sous-chaine *contigu* ; longueur min/max sous contrainte ; fenetre de taille fixe (somme de k elements consecutifs).

Complexite : Temps $O(n)$ | Espace $O(1)$ ou $O(k)$

Problemes types : Longest Substring (LC 3), Min Size Subarray (LC 209), Find Anagrams (LC 438).

```
def longest_unique(s):
    last = {}                                # caractere -> dernier indice vu
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

Piege : bien decider quand *agrandir* (avancer **right**) et quand *retrecir* (avancer **left**) la fenetre ; un ordre incorrect casse l'invariant.

6. Prefix Sum (somme prefixee)

On precalcule `prefix[i] = somme de nums[0..i-1]`. Toute somme d'intervalle se lit alors en $O(1)$: `sum(i, j) = prefix[j+1] - prefix[i]`.

L'idée. Repondre a beaucoup de questions « somme entre i et j » en re-additionnant a chaque fois coute cher. En calculant *une seule fois* les sommes cumulees depuis le debut, toute somme partielle devient une simple soustraction de deux valeurs deja connues. On investit $O(n)$ au depart pour que chaque requete suivante soit gratuite.

Reconnaitre le pattern. *Nombreuses* requetes de somme (ou de compte) sur des intervalles, recherche de sous-tableaux dont la somme vaut une cible, ou notion d'*equilibre* (indice pivot). Tres puissant combine a un hash map : on stocke les sommes prefixees deja vues pour compter les sous-tableaux en un seul passage.

Quand l'utiliser

Nombreuses requetes de somme sur intervalle ; compter les sous-tableaux de somme donnee (combine avec un hash map) ; indice pivot.

Complexite : Temps $O(n)$ precalcul, $O(1)$ requete | Espace $O(n)$

Problemes types : Range Sum Query (LC 303), Subarray Sum = K (LC 560), Pivot Index (LC 724).

```
def subarray_sum_k(nums, k):
    count = running = 0
    seen = {0: 1}                            # somme prefixee -> nb d'occurrences
    for x in nums:
        running += x
        count += seen.get(running - k, 0)
        seen[running] = seen.get(running, 0) + 1
    return count
```

Piege : initialise `seen = {0: 1}` pour compter les sous-tableaux qui commencent a l'indice 0.

7. Sorting (tri)

Trier revele souvent la structure cachee d'un probleme (voisinage, chevauchement). Cout de reference : $O(n \log n)$.

L'idée. Une fois les donnees ordonnees, les elements qui « vont ensemble » deviennent voisins : les doublons se touchent, les intervalles qui se chevauchent se suivent, une recherche dichotomique devient possible. Trier n'est pas la reponse en soi, mais l'*etape preparatoire* qui rend le vrai algorithme simple. En Python, `sort()` (Timsort) est stable et l'argument `key` permet de trier selon n'importe quel critere.

Reconnaitre le pattern. L'ordre des elements n'a pas d'importance dans l'enonce, mais un traitement devient evident *s'ils sont ordonnes* : fusionner des intervalles, detecter des doublons, planifier des reunions, preparer un two-pointers ou une dichotomie. Cout a assumer : $O(n \log n)$.

Quand l'utiliser

Regrouper ou detecter des doublons ; fusionner des intervalles ; preparer un two-pointers ; ordonner selon un critere personnalise.

Complexite : Temps $O(n \log n)$ | Espace $O(n)$ ou $O(1)$

Problemes types : Merge Intervals (LC 56), Meeting Rooms (LC 252), Largest Number (LC 179).

```
def merge_intervals(intervals):
    intervals.sort(key=lambda p: p[0])
    res = []
    for start, end in intervals:
        if res and start <= res[-1][1]:
            res[-1][1] = max(res[-1][1], end)  # fusionner
        else:
            res.append([start, end])
    return res
```

Piege : le tri detruit l'ordre initial. Si tu dois rendre des indices d'origine, trie des paires (valeur, indice).

8. Binary Search (recherche dichotomique)

Recherche en $O(\log n)$ sur un espace trie ou monotone. Cle : trouver un *predicat monotone* (vrai puis faux, ou l'inverse).

L'idée. A chaque etape, on regarde l'element du milieu et on elimine *la moitie* des candidats. Diviser l'espace de recherche par deux a chaque tour donne un cout logarithmique : chercher dans un million d'elements ne demande qu'une vingtaine de comparaisons. La version avantee ne cherche pas une valeur mais la *frontiere* d'une propriete monotone : « quelle est la plus petite valeur qui fonctionne ? ».

Reconnaitre le pattern. Donnees *triees*, ou reponse qui varie de facon *monotone* (si x marche, tout x plus grand marche aussi). Formulation type : « trouver la premiere / derniere position qui... », ou « minimiser une capacite / vitesse telle que... ». Des qu'on peut poser une question a reponse oui/non monotone, la dichotomie s'applique.

Quand l'utiliser

Tableau trie ; reponse monotone (« la valeur x fonctionne-t-elle ? ») ; recherche de la premiere ou derniere position valide.

Complexite : Temps $O(\log n)$ | Espace $O(1)$

Problem types : Binary Search (LC 704), Search in Rotated Array (LC 33), Koko Bananas (LC 875).

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2    # evite le debordement
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Piege : attention aux bornes ($<$ vs $<=$) et a la mise a jour de $\text{mid} \pm 1$ pour eviter les boucles infinies.

9. Stack (pile LIFO)

Last In, First Out. En Python : une simple `list` avec `append()` et `pop()`.

L’idee. La pile retient le dernier element ajoute pour le traiter en premier – exactement le comportement dont on a besoin quand des elements doivent etre « fermes » dans l’ordre inverse de leur ouverture (parentheses, balises), ou quand on veut suspendre une tache pour y revenir plus tard (DFS iteratif, annuler/refaire). Les operations **push** et **pop** sont en $O(1)$.

Reconnaitre le pattern. Correspondances imbriquees (parentheses, accolades, balises), evaluation d’expressions, parcours en profondeur sans recursion, ou tout probleme ou « le plus recent d’abord » est la bonne regle. Signal fort : on doit se souvenir d’elements *en attente* et les traiter dans l’ordre inverse.

Quand l’utiliser

Parentheses / expressions equilibrees ; parsing et evaluation ; DFS iteratif ; fonctionnalites annuler / refaire.

Complexite : Temps $O(1)$ par operation | Espace $O(n)$

Problem types : Valid Parentheses (LC 20), Min Stack (LC 155), Evaluate RPN (LC 150).

```
def valid_parentheses(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for c in s:
        if c in pairs:
            if not stack or stack.pop() != pairs[c]:
                return False
        else:
            stack.append(c)
    return not stack
```

Piege : pense a verifier que la pile est *vide* a la fin ; une pile non vide signale des ouvrants non fermes.

10. Monotonic Stack (pile monotone)

Une pile dont les elements restent toujours croissants ou decroissants. Chaque element est empile et depile au plus une fois : cout total $O(n)$.

L’idée. Quand on cherche, pour chaque element, « le prochain plus grand » (ou plus petit), une double boucle donne $O(n^2)$. La pile monotone garde seulement les candidats encore *utiles* : avant d’empiler un nouvel element, on depile tous ceux qu’il « bat », et c’est justement lui leur reponse. Comme chaque element entre et sort une seule fois, le cout total reste $O(n)$ malgre la boucle interne.

Reconnaitre le pattern. Formulations « prochain / precedent element plus grand / plus petit », spans (combien de jours jusqu’au prochain plus chaud), histogrammes, eaux de pluie. Astuce : on empile souvent des *indices* plutot que des valeurs, pour pouvoir calculer des distances.

Quand l’utiliser

Prochain element plus grand / plus petit ; largest rectangle in histogram ; trapping rain water ; spans boursiers.

Complexite : Temps $O(n)$ | Espace $O(n)$

Problemes types : Daily Temperatures (LC 739), Next Greater Element (LC 496), Largest Rectangle (LC 84).

```
def daily_temperatures(temps):
    res = [0] * len(temps)
    stack = [] # indices, temperatures decroissantes
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res
```

Piege : on empile souvent des *indices* plutot que des valeurs, pour pouvoir calculer des distances.

11. Backtracking (retour sur trace)

Explorer toutes les possibilites via un schema *choisir / recuser / annuler*. Cout souvent exponentiel.

L’idée. On construit une solution pas a pas. A chaque etape on fait un *choix*, on explore recursivement ce que ce choix entraine, puis on *annule* le choix (backtrack) pour essayer le suivant. On explore ainsi un *arbre de decisions*, et l’annulation permet de reutiliser la meme structure pour toutes les branches. On peut couper des branches (pruning) des qu’une solution partielle viole une contrainte, ce qui accelere beaucoup en pratique.

Reconnaitre le pattern. L’enonce demande *toutes* les solutions : combinaisons, permutations, sous-ensembles, ou un placement respectant des contraintes (Sudoku, N-Queens). Signal : « generer / enumerer tous les... ». Attention au cout, naturellement exponentiel ($O(2^n)$ ou $O(n!)$).

Quand l’utiliser

Toutes les combinaisons, permutations ou sous-ensembles ; problemes a contraintes (Sudoku, N-Queens) ; generation exhaustive.

Complexite : Temps $O(2^n)$ ou $O(n!)$ | Espace $O(n)$ pile de recursion

Problemes types : Subsets (LC 78), Permutations (LC 46), Combination Sum (LC 39).

```
def subsets(nums):
    res, path = [], []
    def backtrack(start):
        res.append(path[:]) # copie de l'etat courant
        for i in range(start, len(nums)):
            path.append(nums[i]) # choisir
```

```

        backtrack(i + 1)
        path.pop()          # annuler (backtrack)
    backtrack(0)
    return res

```

Piege : enregistre une *copie* (`path[:]`), sinon toutes les solutions pointeront vers la meme liste mutable.

12. Linked List (liste chaine)

Des noeuds relies par `.val` et `.next`. Les entretiens insistent sur la manipulation propre des pointeurs.

L’idee. Contrairement a un tableau, on ne peut pas sauter directement au *i*-eme element : il faut suivre les liens de proche en proche. Tout le savoir-faire consiste a manipuler quelques pointeurs (`prev`, `curr`, `next`) dans le bon ordre, sans perdre le reste de la liste. Deux techniques reviennent : le *noeud sentinelle* (dummy) qui simplifie la gestion de la tete, et les pointeurs *lent/rapide* pour le milieu ou la detection de cycle.

Reconnaitre le pattern. L’enonce fournit une liste chaine et demande de l’*inverser*, la *fusionner*, detecter un *cycle*, ou supprimer un noeud. Le vrai defi n’est pas l’algorithme mais la *rigueur* sur les cas limites : liste vide, un seul noeud, modification de la tete ou de la queue.

Quand l’utiliser

Inverser une liste ; detecter un cycle (lent/rapide) ; fusionner deux listes ; supprimer le *n*-ieme noeud depuis la fin.

Complexite : Temps $O(n)$ | Espace $O(1)$

Problemes types : Reverse List (LC 206), Linked List Cycle (LC 141), Merge Two Lists (LC 21).

```

def reverse_list(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next          # sauvegarder le suivant
        curr.next = prev        # inverser le lien
        prev = curr
        curr = nxt
    return prev

```

Piege : soigne les cas limites – liste vide, un seul noeud, mise a jour de `head`. Un *noeud sentinelle* (dummy) simplifie souvent le code.

13. Fiche recapitulative des complexites

A memoriser avant l'entretien :

Pattern	Temps	Espace
Hash Map / Set	$O(1)$ moyen	$O(n)$
Two Pointers	$O(n)$	$O(1)$
Sliding Window	$O(n)$	$O(k)$
Prefix Sum	$O(n)$ / $O(1)$	$O(n)$
Sorting	$O(n \log n)$	$O(n)$
Binary Search	$O(\log n)$	$O(1)$
Stack	$O(n)$	$O(n)$
Monotonic Stack	$O(n)$	$O(n)$
Backtracking	$O(2^n)$ / $O(n!)$	$O(n)$
Linked List	$O(n)$	$O(1)$

Checklist du jour J

- Parler a voix haute – le raisonnement est note.
- Clarifier les contraintes AVANT de coder.
- Annoncer la complexite temps + espace.
- Traiter les cas limites (vide, 1 element, negatifs, debordement).
- Nommer clairement ; code lisible > code astucieux.
- Bloque ? Revenir a la force brute, puis optimiser.
- Tester a la main avant de dire « c'est bon ».